# 11

# Применение триггеров

ORACLE

Copyright © 2004, Oracle. All rights reserved.

# Рассматриваемые вопросы

- Создание дополнительных триггеров базы данных
- Правила, управляющие работой триггеров
- Применение триггеров

ORACLE

Copyright © 2004, Oracle. All rights reserved.

**Цели урока**

На этом уроке вы научитесь создавать дополнительные триггеры базы данных и познакомитесь с правилами, управляющими работой триггеров. Вы также узнаете о многих применениях триггеров.

## Создание триггеров базы данных

- **Триггеры, срабатывающие в результате события, определенного пользователем:**
  - **`CREATE`, `ALTER` или `DROP`**
  - **Соединение и отсоединение от базы данных**
- **Триггеры, срабатывающие в результате события в базе данных или системного события:**
  - **Остановка и запуск базы данных**
  - **Возникновение конкретной ошибочной ситуации (или какой-либо ошибки)**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

### Создание триггеров базы данных

Перед кодированием тела триггера необходимо принять решение о структуре триггера.

Триггеры, срабатывающие по *системным событиям* (*system events*), могут быть определены на уровне базы данных или схемы. Например, триггер, прорабатывающий при остановке базы данных, определяется на уровне базы данных. Триггеры, связанные с действиями, вызываемыми командами языка DDL или с соединением и отсоединением пользователей, могут быть определены либо на уровне базы данных, либо на уровне схемы. Триггеры, срабатывающие по командам DML, определяются для таблицы или представления.

Триггер, определенный на уровне базы данных, прорабатывает для всех пользователей, а триггер, определенный для схемы или таблицы, срабатывает только, когда возбуждающее триггер событие затрагивает эту схему или таблицу.

События, по которым могут срабатывать триггеры:

- Команда языка DDL, выполняемая для объекта базы данных или схемы.
- Соединение и отсоединение от базы данных конкретного пользователя (или любого пользователя).
- Остановка и запуск базы данных.
- Возникновение конкретной или какой-либо ошибки.

# Создание триггеров, срабатывающих по командам DDL

## Синтаксис:

```
CREATE [OR REPLACE] TRIGGER имя_триггера
время_срабатывания
[ddl_событие1 [OR ddl_событие2 OR ...]]
ON {DATABASE|SCHEMA}
тело_триггера
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

### Создание триггеров, срабатывающих по командам DDL

| DDL_событие | Описание |
|---|---|
| CREATE | Заставляет сервер Oracle возбуждать триггер всякий раз, когда по команде CREATE новый объект базы данных добавляется в словарь. |
| ALTER | Заставляет сервер Oracle возбуждать триггер всякий раз, когда по команде ALTER изменяются данные об объекте базы данных в словаре. |
| DROP | Заставляет сервер Oracle возбуждать триггер всякий раз, когда по команде DROP данные об объекте базы данных удаляются из словаря. |

Тело триггера представляет собой полный блок PL/SQL.

Вы можете создавать триггеры для этих событий на уровне базы данных (DATABASE) или схемы (SCHEMA). Кроме того, BEFORE или AFTER указываются для задания времени срабатывания.

DDL-триггеры срабатывают только, когда создаются следующие объекты: кластер, функция, индекс, пакет, процедура, роль, последовательность, синоним, таблица, табличное пространство, триггер, тип, представление или пользователь.

# Создание триггеров, срабатывающих по системным событиям

## Синтаксис:

```
CREATE [OR REPLACE] TRIGGER имя_триггера
время_срабатывания
[событие_БД_1 [OR событие_БД_2 OR ...]]
ON {DATABASE|SCHEMA}
тело_триггера
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

**Синтаксис создания триггера**

| *Событие_БД* | **Описание** |
|---|---|
| AFTER SERVERERROR | Заставляет сервер Oracle возбуждать триггер всякий раз, когда генерируется сообщение об ошибке. |
| AFTER LOGON | Заставляет сервер Oracle возбуждать триггер сразу после того, как пользователь установит соединение с базой данных. |
| BEFORE LOGOFF | Заставляет сервер Oracle возбуждать триггер всякий раз перед тем, как пользователь отсоединится от базы. |
| AFTER STARTUP | Заставляет сервер Oracle возбуждать триггер сразу после того, как произойдет открытые базы данных. |
| BEFORE SHUTDOWN | Заставляет сервер Oracle возбуждать триггер всякий раз перед тем, как база данных будет остановлена. |

Вы можете создать триггеры, которые срабатывают после событий, произошедших в базе данных (DATABASE) или в схеме (SCHEMA), за исключением триггеров, вызываемых командами SHUTDOWN и STARTUP, которые применимы только к событиям в базе данных (DATABASE).

# Пример триггеров LOGON и LOGOFF

```
CREATE OR REPLACE TRIGGER logon_trig
AFTER LOGON  ON  SCHEMA
BEGIN
 INSERT INTO log_trig_table(user_id,log_date,action)
 VALUES (USER, SYSDATE, 'Logging on');
END;
/
```

```
CREATE OR REPLACE TRIGGER logoff_trig
BEFORE LOGOFF  ON  SCHEMA
BEGIN
 INSERT INTO log_trig_table(user_id,log_date,action)
 VALUES (USER, SYSDATE, 'Logging off');
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

### Пример триггеров LOGON и LOGOFF

Такие триггеры могут быть созданы для наблюдения за тем, как часто соединяются и отсоединяются пользователи, а также для подготовки сведений о продолжительности сеансов пользователя. Если указана фраза ON SCHEMA, триггер срабатывает для конкретного пользователя, если ON DATABASE – для всех пользователей.

# Оператор CALL

```
CREATE [OR REPLACE] TRIGGER имя_триггера
время_срабатывания
событие1 [OR событие2 OR событие3]
ON имя_таблицы
[REFERENCING OLD AS old | NEW AS new]
[FOR EACH ROW]
[WHEN условие]
CALL имя_процедуры
/
```

```
CREATE OR REPLACE TRIGGER log_employee
BEFORE INSERT ON EMPLOYEES
CALL log_execution
/
```

**Примечание: после оператора CALL точка с запятой не ставится.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

### Оператор CALL

Оператор CALL позволяет вызвать хранимую процедуру и не кодировать тело триггера на PL/SQL . Процедура может быть написана на языках PL/SQL, C или Java.

В параметрах оператора вызова процедуры могут указываться атрибуты :NEW и :OLD, например:

```
CREATE TRIGGER salary_check
     BEFORE UPDATE OF salary, job_id ON employees
     FOR EACH ROW
     WHEN (NEW.job_id <> 'AD_PRES')
     CALL check_salary(:NEW.job_id, :NEW.salary)
     /
```

**Примечание:** после оператора CALL точка с запятой не ставится.

В приведенном примере триггер вызывает процедуру check_sal, которая проверяет, что новый оклад находится в диапазоне допустимых значений для данной должности, заданном в таблице JOBS.

# Чтение данных из мутирующей таблицы

ORACLE
Copyright © 2004, Oracle. All rights reserved.

### Правила, управляющие работой триггеров

Чтение и запись данных с использованием триггеров подчиняются определенным правилам. В соответствие с этими правилами ограничения накладываются только на строчные триггеры, а также на операторный триггер, который срабатывает в результате каскадного удаления (`ON DELETE CASCADE`).

### Мутирующая таблица

*Мутирующая таблица* (*mutating table*) – это либо таблица, обновляемая в данный момент командой `UPDATE`, `DELETE` или `INSERT`, либо таблица, обновление которой может потребоваться вследствие срабатывания каскадного удаления в подчиненной таблице (определенного параметром `DELETE CASCADE` в ссылочном ограничении целостности). К операторным триггерам понятие “мутирующая таблица” не применяется.

Для триггеров, принадлежащих изменяемой таблице, эта таблица является мутирующей. Кроме того, мутирующей является любая другая таблица, ссылающаяся на нее через ограничение `FOREIGN KEY`. Благодаря этому строчный триггер не может прочитать несогласованный набор данных.

# Мутирующая таблица: пример

```
CREATE OR REPLACE TRIGGER check_salary
  BEFORE INSERT OR UPDATE OF salary, job_id
  ON employees
  FOR EACH ROW
  WHEN (NEW.job_id <> 'AD_PRES')
DECLARE
  minsalary employees.salary%TYPE;
  maxsalary employees.salary%TYPE;
BEGIN
  SELECT MIN(salary), MAX(salary)
   INTO  minsalary, maxsalary
   FROM  employees
   WHERE job_id = :NEW.job_id;
  IF :NEW.salary < minsalary OR
     :NEW.salary > maxsalary THEN
     RAISE_APPLICATION_ERROR(-20505,'Out of range');
  END IF;
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

### Мутирующая таблица: пример

Триггер CHECK_SALARY в этом примере пытается гарантировать, что при вставке нового служащего в таблицу EMPLOYEES, а также при изменении оклада или должности существующего служащего новый оклад не выйдет за пределы диапазона окладов для данной должности.

Триггер CHECK_SALARY срабатывает для каждой записи о служащем. В триггере запрашивается та же самая таблица, которая изменяется по операции, вызывающей срабатывание триггера. Поэтому такая таблица – мутирующая таблица.

# Мутирующая таблица: пример

```
UPDATE employees
 SET salary = 3400
 WHERE last_name = 'Stiles';
```

```
UPDATE employees
     *
ERROR at line 1:
ORA-04091: table PLSQL.EMPLOYEES is mutating, trigger/function may not see it
ORA-06512: at "PLSQL.CHECK_SALARY", line 5
ORA-04088: error during execution of trigger 'PLSQL.CHECK_SALARY'
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

### Мутирующая таблица: пример (пример)

В этом примере делается попытка прочитать мутирующую таблицу.

В ходе выполнения проверки того, что оклад находится в определенном диапазоне, в триггере возникает ошибка. Таблица `EMPLOYEES` мутирует, т.е. находится в процессе изменения; следовательно триггер не в состоянии читать ее данные.

К мутации таблицы могут также привести функции, вызываемые из команды DML.

### Возможные решения проблемы

- Храните итоговые данные (минимальный и максимальный оклады) в другой таблице, синхронизируя ее содержимое с помощью других триггеров DML.
- Занесите итоговые данные в пакетные переменные PL/SQL, а затем ими воспользуйтесь. Записать данные в пакетные переменные можно с помощью операторного триггера `BEFORE`.

В зависимости от ситуации решение может быть более сложным и запутанным. Тогда не используйте триггеры базы данных, а рассмотрите возможность реализации чрезмерно сложных бизнес-правил на уровне приложения или на промежуточном уровне.

## Преимущества триггеров базы данных

- **Улучшают защиту данных:**
  - **Сложные проверки обеспечения безопасности**
  - **Дополнительные возможности аудита**
- **Помогают обеспечить целостность данных:**
  - **Динамические проверки соблюдения правил целостности данных**
  - **Проверки соблюдения сложных правил ссылочной целостности**
  - **Обеспечение неявного совместного выполнения взаимосвязанных операций**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

**Преимущества триггеров базы данных**

- Вы можете использовать триггеры базы данных, которые выполняют действия, замещающие возможности, предоставляемые сервером Oracle.
- Разрабатываемые вами триггеры могут в отличие от стандартных возможностей Oracle удовлетворять более сложным или более простым требованиям;
- Вы можете разрабатывать триггеры для удовлетворения требований, которые совсем не поддерживаются сервером Oracle.

# Управление триггерами

**Следующие привилегии необходимы для управления триггерами:**

- **Системные привилегии `CREATE/ALTER/DROP (ANY) TRIGGER` разрешают сопровождение триггеров в любой схеме**
- **Системная привилегия `ADMINISTER DATABASE TRIGGER` позволяет создавать триггер, срабатывающий в базе данных (`DATABASE`)**
- **Привилегия `EXECUTE` для любого объекта, который вызывается из триггера и находится в другой схеме**

**Примечание: команды в теле триггера выполняются в соответствие с правами владельца триггера, а не пользователя, действия которого привели к срабатыванию триггера**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

### Управление триггерами

Для создания триггера в собственной схеме необходимо иметь системную привилегию `CREATE TRIGGER`, а также вы должны быть либо владельцем таблицы, либо иметь объектную привилегию `ALTER` на таблицу, для которой срабатывает триггер, либо иметь системную привилегию `ALTER ANY TABLE`.. Затем вы можете изменять или удалять триггер без каких-либо дополнительных привилегий.

Когда в привилегии присутствует ключевое слово `ANY (CREATE ANY TRIGGER)`, можно создавать, изменять или удалять собственные триггеры, а также триггеры в других схемах, в которых имеются ссылки на любые пользовательские таблицы.

Для того, чтобы сработал триггер в собственной схеме вам не нужны никакие привилегии. Триггер вызывается неявно выполняемой командой DML. Однако если в триггере есть ссылки на любые хранимые программные единицы, не принадлежащие вашей схеме, пользователю необходимо иметь привилегии `EXECUTE` на используемые процедуры, функции и пакеты. Такая привилегия должна быть выделена явно, а не через роль. Как и в случае хранимых процедур, команда в теле триггера выполняется под доменом привилегий владельца триггера, а не пользователя, вызвавшего команду, по которой произошло срабатывание триггера.

Для создания триггера типа `DATABASE` (срабатывают после событий в базе данных) необходимо иметь привилегию `ADMINISTER DATABASE TRIGGER`. После того, как эта привилегия в дальнейшем отбирается, вы можете удалить, но не изменить триггер.

# Практическое применение триггеров

- Защита
- Аудит
- Целостность данных
- Ссылочная целостность
- Репликация таблиц
- Автоматическое вычисление производных данных
- Регистрация событий

**Примечание: в приложении C более подробно рассмотрены примеры такого применения триггеров.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

## Практическое применение триггеров

*Триггеры базы данных* (*database triggers*) разрабатываются для расширения функциональных возможностей, если это нельзя реализовать в Oracle иным способом, а также для замены возможностей, предоставляемых сервером Oracle.

| Функция | Дополнительные возможности |
|---|---|
| Защита | Oracle предоставляет право доступа к таблицам отдельным пользователям и ролям. Триггеры разрешают доступ в зависимости от значений данных. |
| Аудит | Oracle отслеживает операции над данными таблиц. Триггеры отслеживают значения при выполнении операций над данными таблиц. |
| Целостность данных | Oracle обеспечивает соблюдение декларативных ограничений. Триггеры позволяют внедрить сложные правила целостности. |
| Ссылочная целостность | Oracle обеспечивает соблюдение стандартных правил ссылочной целостности. Триггеры предоставляют нестандартные функциональные возможности. |
| Репликация таблиц | Oracle асинхронно копирует таблицы в моментальные снимки (snapshots). Триггеры синхронно реплицируют (копируют) данные таблиц. |
| Производные данные | Можно вручную задавать вычисления, производимые сервером Oracle. Триггеры вычисляют производные значения данных автоматически. |
| Регистрация событий | Oracle явно регистрирует события в журнале, триггеры – незаметно для пользователя или программы. |

## Просмотр информации о триггерах

- Представление словаря данных `USER_OBJECTS` содержит сведения об объектах, принадлежащих пользователю
- Представление словаря данных `USER_TRIGGERS` показывает текст триггера
- Представление словаря данных `USER_ERRORS` выводит синтаксические ошибки PL/SQL (ошибки компиляции) триггера

ORACLE

Copyright © 2004, Oracle. All rights reserved.

### Просмотр информации о триггерах

На слайде приведены представления словаря данных, к которым можно обратиться для получения сведений о триггерах.

Представление `USER_OBJECTS` содержит имя и статус триггера, а также дату и время создания триггера.

Представление `USER_ERRORS` выводит подробные сведения об ошибках, которые были обнаружены при компиляции триггера. Содержимое этих представлений похоже на сведения, выдаваемые о подпрограммах.

Представление `USER_TRIGGERS` содержит такие подробные данные, как имя, тип, событие, по которому срабатывает триггер, таблица, для которой создан триггер, а также тело триггера.

Следующая команда в теле триггера выводит имя владельца триггера, а не пользователя, изменяющего таблицу:

```
SELECT Username FROM USER_USERS;
```

# Использование представления USER_TRIGGERS

| Столбец | Описание столбца |
|---|---|
| TRIGGER_NAME | Имя триггера |
| TRIGGER_TYPE | Тип: BEFORE, AFTER, INSTEAD OF |
| TRIGGERING_EVENT | Операция DML, вызывающая срабатывание триггера |
| TABLE_NAME | Имя таблицы базы данных |
| REFERENCING_NAMES | Имена для квалификаторов :OLD и :NEW |
| WHEN_CLAUSE | Используемое when_предложение |
| STATUS | Статус триггера |
| TRIGGER_BODY | Действие, производимое триггером |

*Сокращенный список столбцов

ORACLE

Copyright © 2004, Oracle. All rights reserved.

### Использование представления USER_TRIGGERS

В случае недоступности файла с исходным кодом триггера, можно получить его в *i*SQL*Plus из представления USER_TRIGGERS. Кроме того, можно использовать представления ALL_TRIGGERS и DBA_TRIGGERS, в каждом из которых имеется столбец OWNER, указывающий на владельца объекта.

# Распечатка кода триггеров

```
SELECT trigger_name, trigger_type, triggering_event,
        table_name, referencing_names,
        status, trigger_body
FROM   user_triggers
WHERE  trigger_name = 'RESTRICT_SALARY';
```

| TRIGGER_NAME | TRIGGER_TYPE | TRIGGERING_EVENT | TABLE_NAME | REFERENCING_NAMES | WHEN_CLAUS | STATUS | TRIGGER_BODY |
|---|---|---|---|---|---|---|---|
| RESTRICT_SALARY | BEFORE EACH ROW | INSERT OR UPDATE | EMPLOYEES | REFERENCING NEW AS NEW OLD AS OLD | | ENABLED | BEGIN IF NOT (:NEW.JOB_ ID IN ('AD_PRES ', 'AD_VP')) AND :NE W.SAL |

Copyright © 2004, Oracle. All rights reserved.

**Пример**

Для вывода информации о триггере `RESTRICT_SALARY` используется представление словаря данных `USER_TRIGGERS`.

# Итоги

- Использование дополнительных триггеров базы данных
- Правила, вызывающие мутирование таблиц, используемых триггерами
- Различные применения триггеров
- Управление триггерами
- Просмотр информации о триггерах

ORACLE

Copyright © 2004, Oracle. All rights reserved.

# Обзор практического занятия 11

- **Создание триггеров, реализующих дополнительные правила управления целостностью данных.**
- **Создание триггеров, вызывающих ошибку мутации таблицы.**
- **Создание триггеров, использующих состояние пакета для решения проблемы мутирующей таблицы.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

### Обзор практического занятия 11

На этом практическом занятии вы с помощью триггера реализуете простое бизнес-правило, относящиеся к целостности данных о размере оклада служащих. Для этого вы воспользуетесь таблицей с допустимыми диапазонами окладов для каждой должности.

Ваш новый триггер вызовет каскадный выполнение триггеров, созданных на предыдущем практическом занятии. Каскадный эффект повлечет за собой ошибку, вызванную мутацией таблицы `JOBS`. Поэтому затем вы создадите пакет PL/SQL и дополнительные триггеры, решающие проблему мутации таблицы.

**Практическое занятие 11**

1. Сотрудники должны получать автоматическую прибавку к зарплате, когда минимальный оклад для должности увеличивается и становится больше текущего оклада сотрудника. Реализуйте это требование с помощью пакетной процедуры, вызываемой из триггера таблицы JOBS. При изменении минимального оклада в таблице JOBS и обновлении оклада служащих в триггере CHECK_SALARY выполняется попытка чтения таблицы JOBS, которая сама изменяется. Поэтому возникает исключение, вызванное мутацией таблицы. Эта проблеме решается путем создания нового пакета и дополнительных триггеров.

   (а). Измените созданный вами на практическом занятии 7 пакет EMP_PKG. Добавьте в него процедуру SET_SALARY, которая изменяет оклады служащих. Эта процедура принимает два параметра:идентификатор должности сотрудников, которым может быть повышен оклад, и новый минимальный оклад для этой должности. Если оклады сотрудников ниже нового минимального значения для их должности, тогда процедура устанавливает этим сотрудникам новый оклад, равный минимальному.

   (б). Создайте строчный триггер UPD_MINSALARY_TRG для таблицы JOBS, из которого вызывается процедура EMP_PKG.SET_SALARY, когда для определенного идентификатора должности изменяется минимальный оклад в таблице JOBS.

   (в). Напишите запрос, по которому для всех программистов (JOB_ID = 'IT_PROG') выводится номер служащего, фамилия и текущий оклад. Затем измените минимальный оклад программистов в таблице JOBS, увеличив его на $1000. Что произошло?

2. Чтобы устранить мутацию, создайте пакет JOBS_PKG, сопровождающий в памяти копию строк таблицы JOBS. В процедуре CHECK_SALARY используйте пакетные переменные вместо запрашиваемых из таблицы данных. Это позволит избежать ошибки мутации. Кроме того, необходимо создать операторный триггер BEFORE INSERT OR UPDATE для таблицы EMPLOYEES, в котором инициализируется состояние пакета JOBS_PKG до того, как срабатывает строчный триггер CHECK_SALARY.

   (а). Создайте новый пакет JOBS_PKG со следующей спецификацией:

```
PROCEDURE initialize;
FUNCTION get_minsalary(jobid VARCHAR2) RETURN NUMBER;
FUNCTION get_maxsalary(jobid VARCHAR2) RETURN NUMBER;
PROCEDURE set_minsalary(jobid VARCHAR2,min_salary
NUMBER);
PROCEDURE set_maxsalary(jobid VARCHAR2,max_salary
NUMBER);
```

   (б). Разработайте тело пакета JOBS_PKG, в котором:
   - объявляется частная PL/SQL-таблица (INDEX BY) с именем jobs_tabtype, индексируемая с помощью строки с типом JOBS.JOB_ID%TYPE;
   - объявляется частная переменная jobstab с типом jobs_tabtype.

В процедуре INITIALIZE производится чтение строк таблицы JOBS. При этом используется цикл с курсором и значение JOB_ID устанавливается в качестве индекса PL/SQL-таблицы jobstab.
Функция GET_MINSALARY использует параметр jobid в качестве индекса для jobstab и возвращает min_salary для переданной должности.
Функция GET_MAXSALARY использует параметр jobid в качестве индекса для jobstab и возвращает max_salary для переданной должности.
В процедуре SET_MINSALARY значение параметра jobid используется в качестве индекса для jobstab. Это делается для того, чобы в поле min_salary элемента PL/SQL-таблицы установить значение, равное параметру min_salary.
В процедуре SET_MAXSALARY подобным же образом используется параметр jobid, чтобы поле max_salary элемента PL/SQL-таблицы jobstab установить значение, равное параметру max_salary.

(в). Скопируйте процедуру CHECK_SALARY, созданную при выполнении упражнения 1(а) практического занятия 10. Измените код, заменив в нем запрос к таблице JOBS на команды задания значений переменных minsal и maxsal. Используйте для этого вызовы соответствующих пакетных функций GET_*SALARY, возвращающих данные из пакета JOBS_PKG. Этот шаг устраняет исключение в триггере, вызванное мутацией.

(г). Создайте операторный триггер BEFORE INSERT OR UPDATE с именем INIT_JOBPKG_TRG, содержащий оператор CALL для вызова процедуры JOBS_PKG.INITIALIZE. Такой триггер гарантирует соответствие пакетного состояния на момент времени перед выполнением операции DML.

(д). Проверьте измененный код. Для этого выполните запрос для вывода сведений о заработной плате программистов (IT_PROG), затем введите команду update для увеличения их минимального оклада на 1000 в таблице JOBS, после чего снова запросите сведения о заработной плате, чтобы проверить результаты. Оклад каких программистов стал равен минимальному окладу для этой должности?

3. Процедура CHECK_SALARY вызывается из триггера CHECK_SALARY_TRG, который срабатывает перед вставкой или изменением строки о сотруднике. Необходимо проверить, будет ли эта процедура отрабатывать ожидаемым образом.

(а). Для проверки добавьте нового сотрудника, используя процедуру EMP_PKG.ADD_EMPLOYEE со следующими параметрами: ('Steve', 'Morse', 'SMORSE', sal => 6500). Что произошло?

(б). Для устранения проблемы, выявленной при добавлении и изменении сведений о сотруднике, создайте для таблицы EMPLOYEES операторный триггер BEFORE INSERT OR UPDATE с именем EMPLOYEE_INITJOBS_TRG. В триггере с помощью оператора CALL вызывается процедура JOBS_PKG.INITIALIZE.

(в). Чтобы проверить триггер, еще раз попытайтесь добавить сотрудника Steve Morse. Удостоверьтесь в том, что вставка прошла успешно. Для этого выведите номер служащего, его имя и фамилию, оклад, идентификатор должности и номер отдела.